

Advanced Data Science Lab [MCA33PE21]

Formative Assessment – 01

Installation, Python Programming and Statistical Concepts

Field	Details
Academic Year	2026-2027
Class / Division	SYMCA (Semester-I)
PRN	125M1H026
Student Name	Shantanu Suryawanshi
Submission Date	02-09-206

Log Transformation & Scaling

Dataset: Ames Housing / House Prices

Target: `SalePrice`

Objective

Study the effect of log transformation and feature scaling on a simple Linear Regression model.

Required workflow

- Load and explore data
- Visualize distributions, relationships and outliers
- Handle suitable extreme values
- Split into training/testing sets
- Train Linear Regression
- Evaluate using R^2 , **MAE**, **MSE** and **RMSE**
- Visualize a regression line
- Repeat the pipeline after log transformation + StandardScaler
- Compare both experiments
- Save outputs for a Streamlit dashboard

The final conclusion must be based on measured results; improvement must not be assumed.

▼ 1. Import Libraries

We use pandas for data handling, NumPy for numerical operations, Matplotlib for visualization, and scikit-learn for preprocessing, regression and evaluation.

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.linear_model import LinearRegression
9 from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
10
11 pd.set_option("display.max_columns", 100)
12 os.makedirs("../outputs", exist_ok=True)
```

2. Load the Dataset

The notebook is inside `notebooks/`, so the Kaggle training file is loaded from `../data/train.csv`.

```
1 data_file = "../data/train.csv"
2 df = pd.read_csv(data_file)
3
4 print("Dataset loaded successfully.")
5 print("Rows:", df.shape[0])
6 print("Columns:", df.shape[1])
```

```
Dataset loaded successfully.
Rows: 1460
Columns: 81
```

3. Explore the Dataset

First inspect records, dimensions, data types, missing values and descriptive statistics.

```
1 df.head()
```

```
1 df.shape
```

```
(1460, 81)
```

```
1 df.info()
```

```
1 missing_values = df.isnull().sum().sort_values(ascending=False)
2 missing_values[missing_values > 0].head(20)
```

```
1 df.describe().T
```

4. Select Features and Target

`SalePrice` is the regression target. We use a maximum of ten meaningful numerical predictors so the experiment remains interpretable.

The final transformation decision is based on the observed distributions rather than assumptions.

```
1 candidate_features = [
2     "OverallQual", "GrLivArea", "GarageCars", "GarageArea",
3     "TotalBsmtSF", "1stFlrSF", "FullBath", "YearBuilt",
4     "YearRemodAdd", "TotRmsAbvGrd"
5 ]
6 target_column = "SalePrice"
7
8 model_data = df[candidate_features + [target_column]].copy()
9 model_data.head()
```

5. Handle Missing Values

For numerical predictors, median imputation is robust to extreme values. Rows with a missing target are removed because they cannot provide a training label.

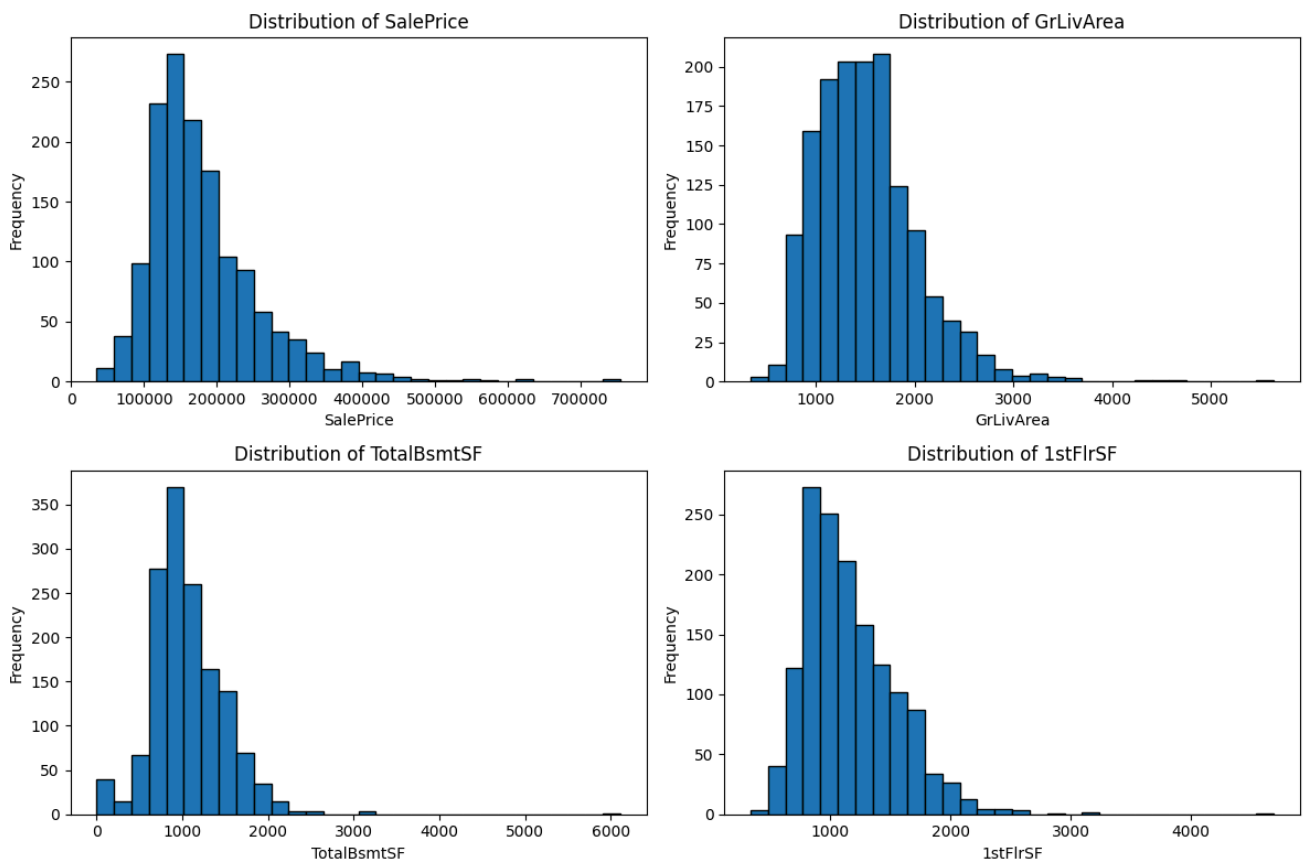
```
1 print(model_data[candidate_features].isnull().sum()[lambda x: x > 0])
2
3 for column in candidate_features:
4     if model_data[column].isnull().any():
5         model_data[column] = model_data[column].fillna(model_data[column].median())
6
7 model_data = model_data.dropna(subset=[target_column]).copy()
8
9 print("Remaining rows:", len(model_data))
10 print("Remaining missing values:", model_data.isnull().sum().sum())
```

6. Measure Skewness

Skewness provides numerical evidence of asymmetric distributions. Histograms below provide the corresponding visual evidence.

```
1 skewness_before = model_data[candidate_features + [target_column]].skew().sort_values(ascending=False)
2 skewness_before.to_frame("Skewness")
```

```
1 plot_columns = ["SalePrice", "GrLivArea", "TotalBsmtSF", "1stFlrSF"]
2
3 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
4 for axis, column in zip(axes.ravel(), plot_columns):
5     axis.hist(model_data[column], bins=30, edgecolor="black")
6     axis.set_title(f"Distribution of {column}")
7     axis.set_xlabel(column)
8     axis.set_ylabel("Frequency")
9
10 plt.tight_layout()
11 plt.savefig("../outputs/original_distributions.png", dpi=150, bbox_inches="tight")
12 plt.show()
```



7. Box Plots and Outliers

The IQR rule identifies potential outliers. A potential outlier is not automatically an incorrect observation; in housing data, unusually large houses can be legitimate.

```
1 plt.figure(figsize=(12, 6))
2 model_data[candidate_features + [target_column]].boxplot(rot=45)
3 plt.title("Box Plot of Selected Variables")
4 plt.ylabel("Value")
5 plt.tight_layout()
6 plt.savefig("../outputs/boxplots_before_outlier_handling.png", dpi=150, bbox_inches="tight")
7 plt.show()
```

```

1 def count_iqr_outliers(series):
2     q1 = series.quantile(0.25)
3     q3 = series.quantile(0.75)
4     iqr = q3 - q1
5     lower = q1 - 1.5 * iqr
6     upper = q3 + 1.5 * iqr
7     return ((series < lower) | (series > upper)).sum()
8
9 outlier_counts = pd.Series({
10     column: count_iqr_outliers(model_data[column])
11     for column in candidate_features + [target_column]
12 }).sort_values(ascending=False)
13
14 outlier_counts.to_frame("Potential IQR Outliers")

```

8. Outlier Treatment

We use IQR capping for selected size-related variables. Capping retains the observations while limiting the influence of extreme magnitudes.

We do **not** automatically delete every outlier because extreme house characteristics may be valid observations.

```

1 def iqr_cap(series):
2     q1 = series.quantile(0.25)
3     q3 = series.quantile(0.75)
4     iqr = q3 - q1
5     lower = q1 - 1.5 * iqr
6     upper = q3 + 1.5 * iqr
7     return series.clip(lower=lower, upper=upper)
8
9 clean_data = model_data.copy()
10 cap_columns = ["GrLivArea", "TotalBsmtSF", "1stFlrSF"]
11
12 for column in cap_columns:
13     clean_data[column] = iqr_cap(clean_data[column])
14
15 outlier_counts_after = pd.Series({
16     column: count_iqr_outliers(clean_data[column])
17     for column in candidate_features + [target_column]
18 }).sort_values(ascending=False)
19
20 pd.DataFrame({"Before": outlier_counts, "After": outlier_counts_after})

```

```

1 plt.figure(figsize=(12, 6))
2 clean_data[candidate_features + [target_column]].boxplot(rot=45)
3 plt.title("Box Plot After Selected IQR Capping")
4 plt.ylabel("Value")
5 plt.tight_layout()
6 plt.savefig("../outputs/boxplots_after_outlier_handling.png", dpi=150, bbox_inches="tight")
7 plt.show()

```

9. Feature Comparison with SalePrice

A correlation view helps us understand which selected variables have stronger linear relationships with the target. Correlation is exploratory evidence, not proof of causation.

```

1 correlation_with_target = (
2     clean_data[candidate_features + [target_column]]
3     .corr()[target_column]
4     .drop(target_column)
5     .sort_values(key=np.abs, ascending=False)
6 )
7 correlation_with_target.to_frame("Correlation with SalePrice")

```

```

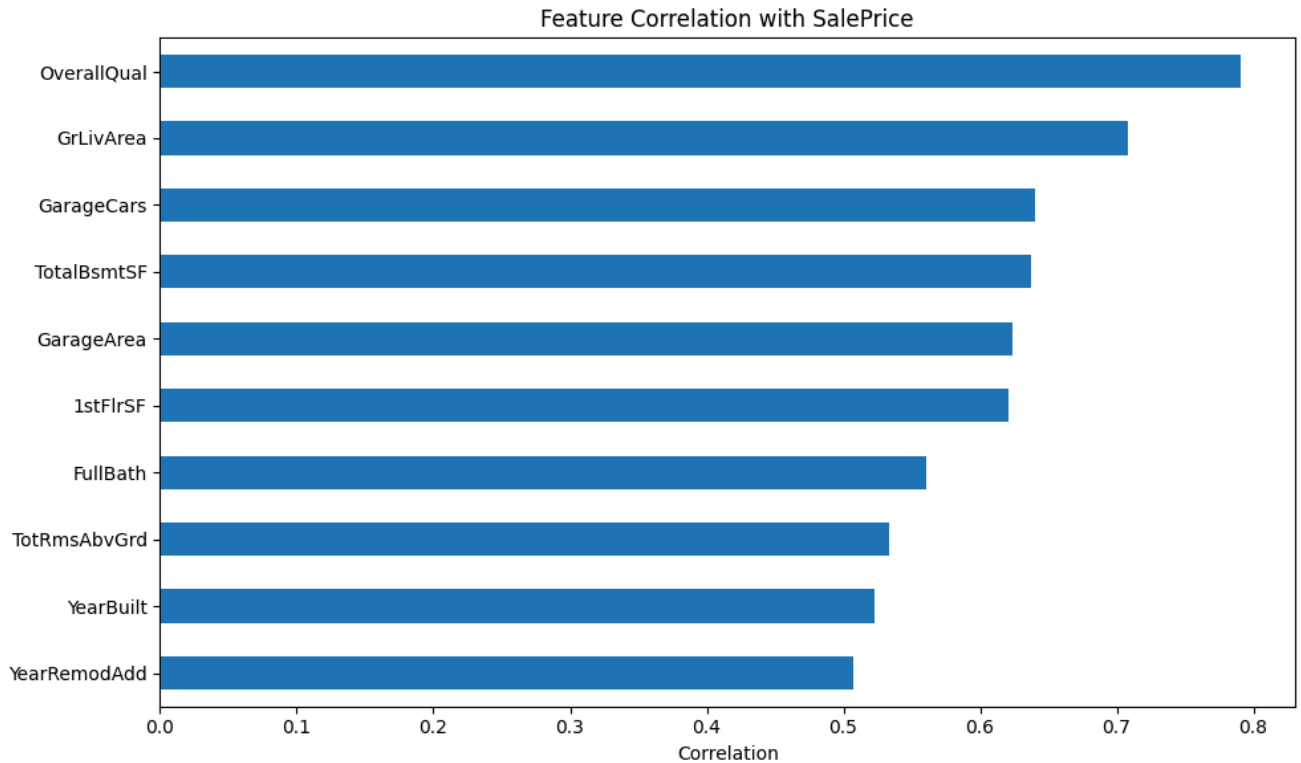
1 plt.figure(figsize=(10, 6))
2 correlation_with_target.sort_values().plot(kind="barh")
3 plt.title("Feature Correlation with SalePrice")

```

```

4 plt.xlabel("Correlation")
5 plt.tight_layout()
6 plt.savefig("../outputs/feature_correlation.png", dpi=150, bbox_inches="tight")
7 plt.show()

```



✂ 10. Train/Test Split — Baseline

We use the same 80/20 split for both experiments so that the comparison is fair and reproducible.

```

1 X = clean_data[candidate_features].copy()
2 y = clean_data[target_column].copy()
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.20, random_state=42
6 )
7
8 print("Training rows:", len(X_train))
9 print("Testing rows :", len(X_test))

```

```

Training rows: 1168
Testing rows : 292

```

🚗 11. Train Baseline Linear Regression

The baseline model uses the selected features without log transformation or scaling.

```

1 baseline_model = LinearRegression()
2 baseline_model.fit(X_train, y_train)
3 baseline_predictions = baseline_model.predict(X_test)

```

🔧 12. Evaluate Baseline Model

Required metrics: R^2 , MAE, MSE and RMSE.

```

1 baseline_r2 = r2_score(y_test, baseline_predictions)
2 baseline_mae = mean_absolute_error(y_test, baseline_predictions)

```

```

3 baseline_mse = mean_squared_error(y_test, baseline_predictions)
4 baseline_rmse = np.sqrt(baseline_mse)
5
6 baseline_results = pd.DataFrame({
7     "Metric": ["R²", "MAE", "MSE", "RMSE"],
8     "Before Transformation": [
9         baseline_r2, baseline_mae, baseline_mse, baseline_rmse
10    ]
11 })
12 baseline_results

```

✓ 13. Baseline Actual vs Predicted

For a multiple regression model, there is no single 2D regression line. This actual-vs-predicted plot is therefore the primary model visualization.

```

1 plt.figure(figsize=(8, 6))
2 plt.scatter(y_test, baseline_predictions, alpha=0.6)
3
4 low = min(y_test.min(), baseline_predictions.min())
5 high = max(y_test.max(), baseline_predictions.max())
6
7 plt.plot([low, high], [low, high], linestyle="--")
8 plt.title("Baseline: Actual vs Predicted SalePrice")
9 plt.xlabel("Actual SalePrice")
10 plt.ylabel("Predicted SalePrice")
11 plt.tight_layout()
12 plt.savefig("../outputs/baseline_actual_vs_predicted.png", dpi=150, bbox_inches="tight")
13 plt.show()

```

✓ 14. Regression Line Visualization

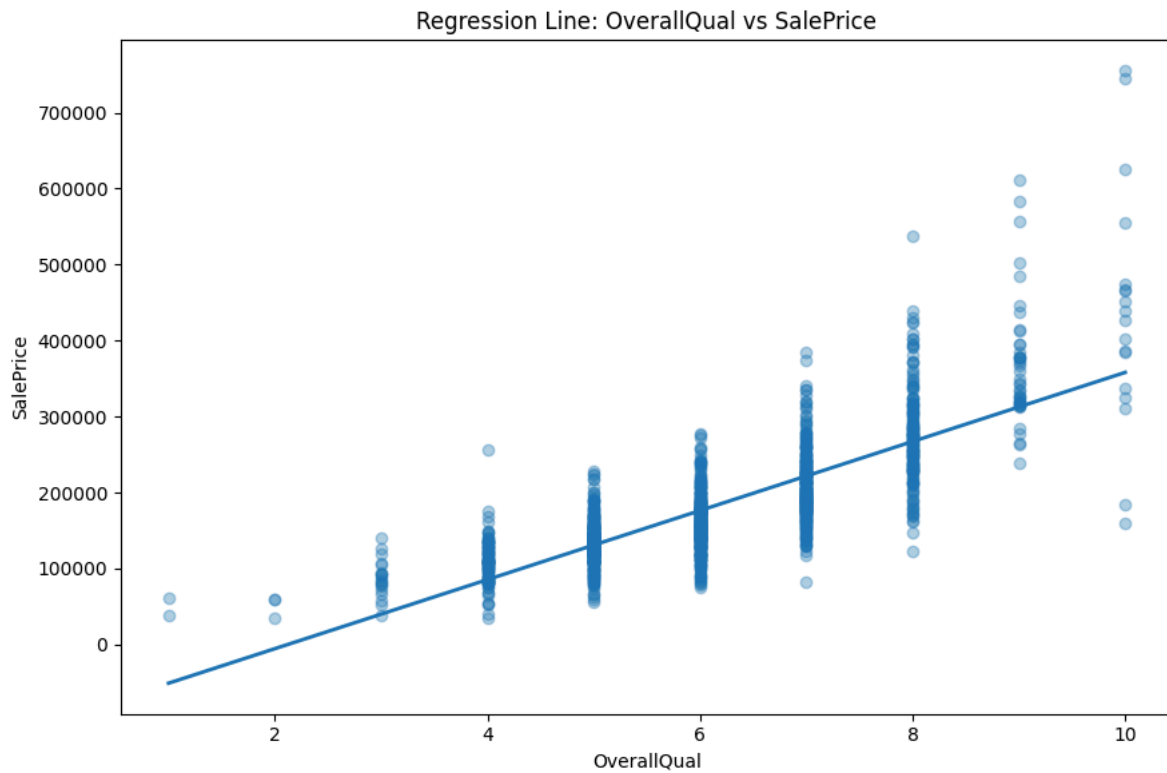
Because a multiple regression model has many dimensions, we use the strongest correlated feature for a simple 2D regression-line visualization. This is an explanatory plot, separate from the full baseline model.

```

1 main_feature = correlation_with_target.abs().idxmax()
2
3 line_model = LinearRegression()
4 line_model.fit(clean_data[[main_feature]], clean_data[target_column])
5
6 line_x = np.linspace(
7     clean_data[main_feature].min(),
8     clean_data[main_feature].max(),
9     200
10 ).reshape(-1, 1)
11 line_y = line_model.predict(line_x)
12
13 plt.figure(figsize=(9, 6))
14 plt.scatter(clean_data[main_feature], clean_data[target_column], alpha=0.35)
15 plt.plot(line_x, line_y, linewidth=2)
16 plt.title(f"Regression Line: {main_feature} vs SalePrice")
17 plt.xlabel(main_feature)
18 plt.ylabel("SalePrice")
19 plt.tight_layout()
20 plt.savefig("../outputs/regression_line.png", dpi=150, bbox_inches="tight")
21 plt.show()
22
23 print("Feature used:", main_feature)

```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
warnings.warn(
```



Feature used: OverallQual

15. Log Transformation

The transformed experiment applies $\log_{1p}(x) = \log(1 + x)$ to selected positive, skewed predictors.

The target remains on the original `SalePrice` scale so both experiments can be compared in the same units.

```
1 transformed_data = clean_data.copy()
2
3 log_columns = ["GrLivArea", "TotalBsmtSF", "1stFlrSF"]
4 log_columns = [column for column in log_columns if column in transformed_data.columns]
5
6 for column in log_columns:
7     transformed_data[column] = np.log1p(transformed_data[column])
8
9 print("Log-transformed columns:", log_columns)
```

Log-transformed columns: ['GrLivArea', 'TotalBsmtSF', '1stFlrSF']

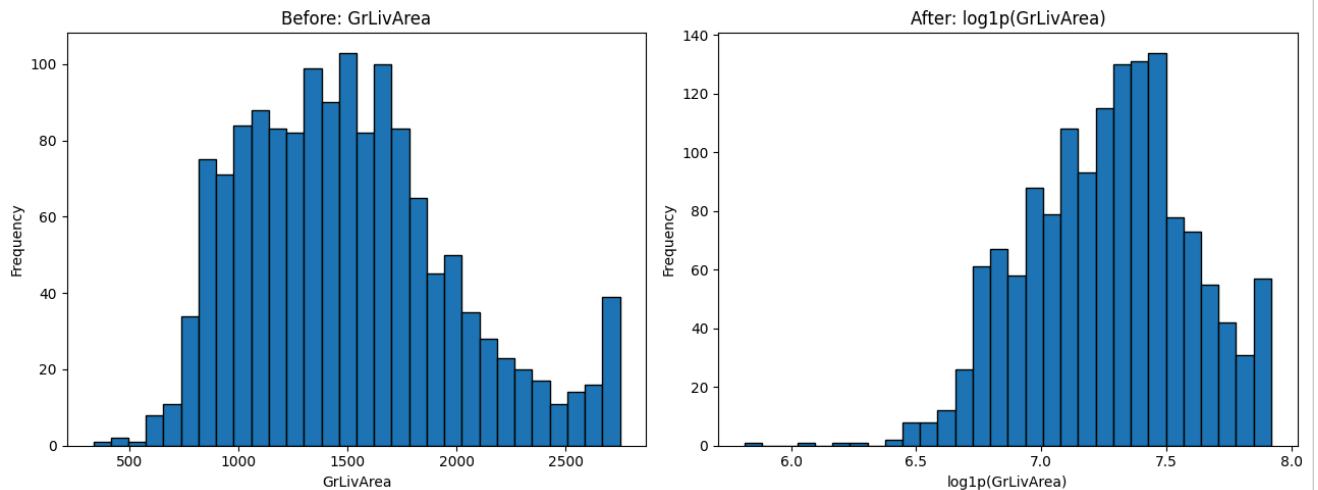
16. Compare Skewness Before and After

```
1 skewness_after = transformed_data[candidate_features].skew()
2
3 skewness_comparison = pd.DataFrame({
4     "Before": clean_data[candidate_features].skew(),
5     "After": skewness_after
6 })
7
8 skewness_comparison["Absolute Skewness Change"] = (
9     skewness_comparison["Before"].abs()
10    - skewness_comparison["After"].abs()
11 )
12
13 skewness_comparison.sort_values("Absolute Skewness Change", ascending=False)
```

```

1 comparison_feature = log_columns[0]
2
3 fig, axes = plt.subplots(1, 2, figsize=(13, 5))
4
5 axes[0].hist(clean_data[comparison_feature], bins=30, edgecolor="black")
6 axes[0].set_title(f"Before: {comparison_feature}")
7 axes[0].set_xlabel(comparison_feature)
8 axes[0].set_ylabel("Frequency")
9
10 axes[1].hist(transformed_data[comparison_feature], bins=30, edgecolor="black")
11 axes[1].set_title(f"After: log1p({comparison_feature})")
12 axes[1].set_xlabel(f"log1p({comparison_feature})")
13 axes[1].set_ylabel("Frequency")
14
15 plt.tight_layout()
16 plt.savefig("../outputs/before_and_after_log.png", dpi=150, bbox_inches="tight")
17 plt.show()

```



17. StandardScaler

The scaler is fitted **only on the training data** and then reused on the test data. This prevents test-set information from leaking into model training.

```

1 X_transformed = transformed_data[candidate_features]
2 y_transformed = transformed_data[target_column]
3
4 X_train_transformed = X_transformed.loc[X_train.index]
5 X_test_transformed = X_transformed.loc[X_test.index]
6
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train_transformed)
9 X_test_scaled = scaler.transform(X_test_transformed)

```

18. Train Transformed Linear Regression

```

1 transformed_model = LinearRegression()
2 transformed_model.fit(X_train_scaled, y_train)
3 transformed_predictions = transformed_model.predict(X_test_scaled)

```

19. Evaluate Transformed Model

```

1 transformed_r2 = r2_score(y_test, transformed_predictions)
2 transformed_mae = mean_absolute_error(y_test, transformed_predictions)
3 transformed_mse = mean_squared_error(y_test, transformed_predictions)
4 transformed_rmse = np.sqrt(transformed_mse)

```

```

5
6 transformed_results = pd.DataFrame({
7     "Metric": ["R2", "MAE", "MSE", "RMSE"],
8     "After Log + Scaling": [
9         transformed_r2, transformed_mae, transformed_mse, transformed_rmse
10    ]
11 })
12 transformed_results

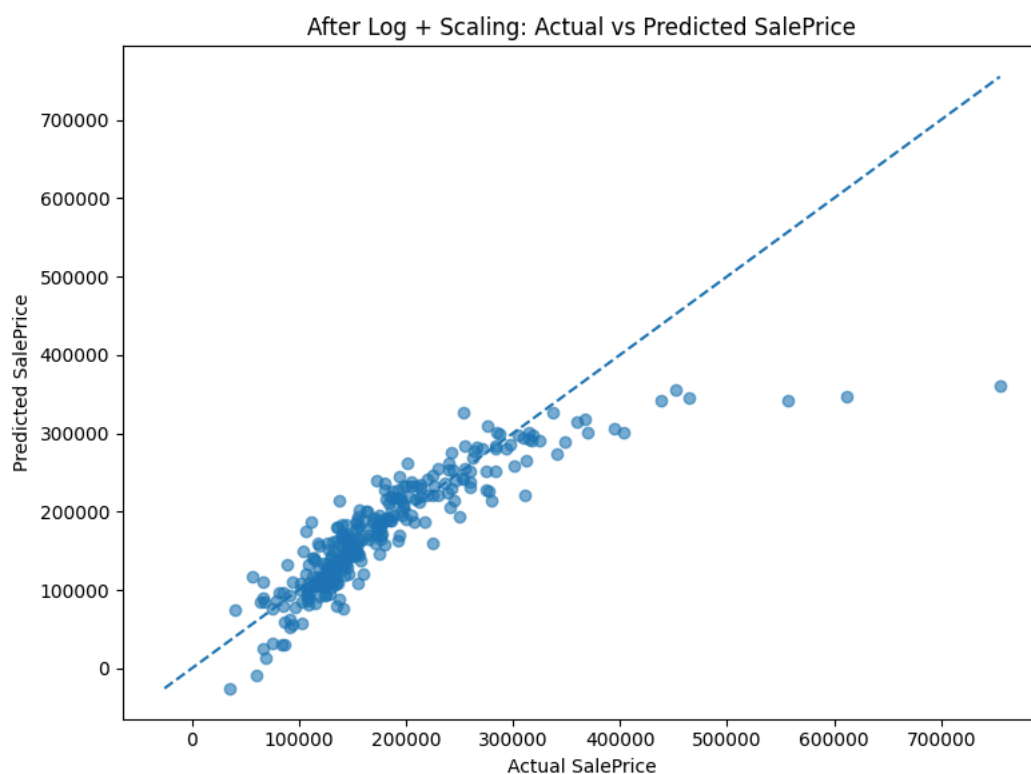
```

20. Transformed Actual vs Predicted

```

1 plt.figure(figsize=(8, 6))
2 plt.scatter(y_test, transformed_predictions, alpha=0.6)
3
4 low = min(y_test.min(), transformed_predictions.min())
5 high = max(y_test.max(), transformed_predictions.max())
6
7 plt.plot([low, high], [low, high], linestyle="--")
8 plt.title("After Log + Scaling: Actual vs Predicted SalePrice")
9 plt.xlabel("Actual SalePrice")
10 plt.ylabel("Predicted SalePrice")
11 plt.tight_layout()
12 plt.savefig("../outputs/transformed_actual_vs_predicted.png", dpi=150, bbox_inches="tight")
13 plt.show()

```



21. Compare Both Models

Both experiments use the same observations, target, split and evaluation metrics. The comparison isolates the effect of the preprocessing pipeline.

```

1 comparison = pd.DataFrame({
2     "Metric": ["R2", "MAE", "MSE", "RMSE"],
3     "Before Transformation": [
4         baseline_r2, baseline_mae, baseline_mse, baseline_rmse
5     ],
6     "After Log + Scaling": [
7         transformed_r2, transformed_mae, transformed_mse, transformed_rmse
8     ]
9 })

```

```

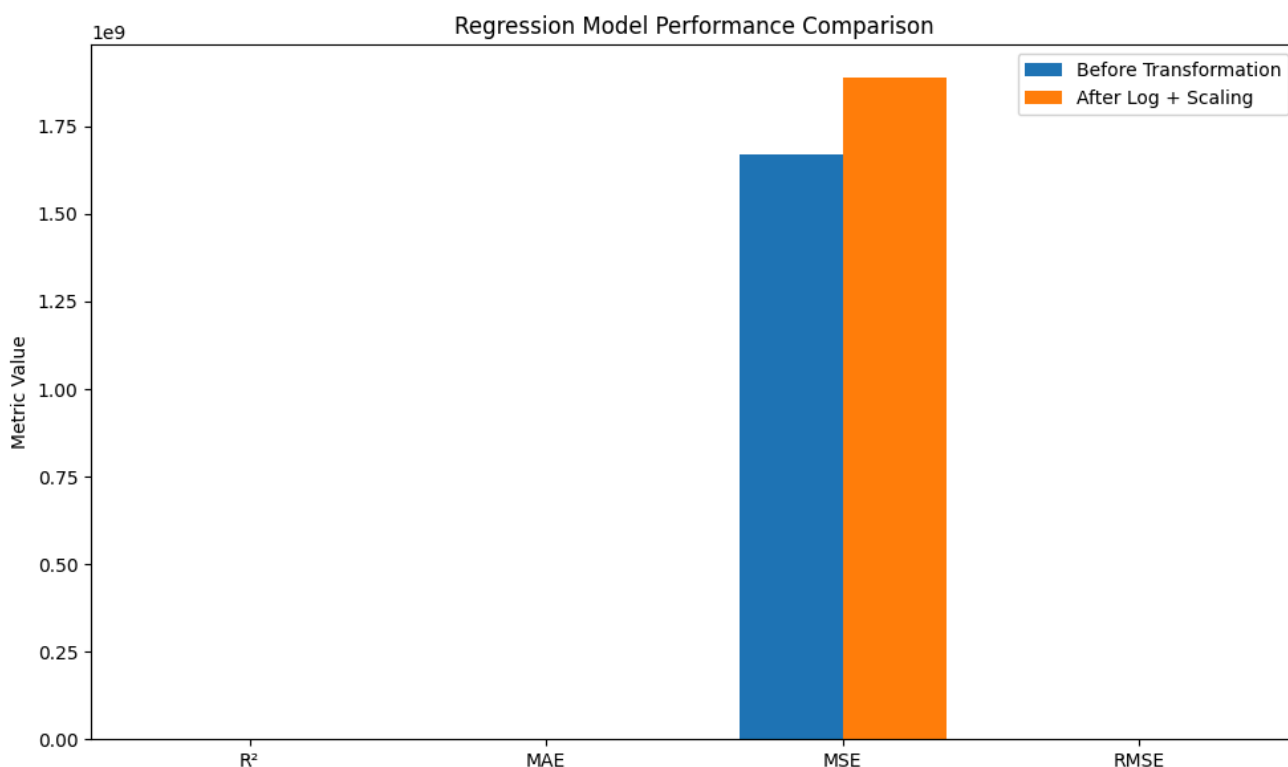
10
11 comparison.to_csv("../outputs/model_comparison.csv", index=False)
12 comparison

```

```

1 plt.figure(figsize=(10, 6))
2 x = np.arange(len(comparison["Metric"]))
3 width = 0.35
4
5 plt.bar(x - width/2, comparison["Before Transformation"], width,
6         label="Before Transformation")
7 plt.bar(x + width/2, comparison["After Log + Scaling"], width,
8         label="After Log + Scaling")
9
10 plt.xticks(x, comparison["Metric"])
11 plt.title("Regression Model Performance Comparison")
12 plt.ylabel("Metric Value")
13 plt.legend()
14 plt.tight_layout()
15 plt.savefig("../outputs/model_comparison.png", dpi=150, bbox_inches="tight")
16 plt.show()

```



22. Percentage Change

For MAE, MSE and RMSE, a negative percentage change means the error decreased. For R^2 , a positive change generally indicates improvement.

```

1 summary = comparison.copy()
2 summary["Change (%)"] = (
3     (summary["After Log + Scaling"] - summary["Before Transformation"])
4     / summary["Before Transformation"].replace(0, np.nan)
5 ) * 100
6 summary

```

23. Final Model Accuracy Comparison

The following comparison shows the performance of both regression models using the R^2 score.

- **Before Transformation:** Linear Regression using the original feature values.

- **After Log + Scaling:** Linear Regression using log-transformed features followed by StandardScaler.

A higher R^2 score indicates that the model explains a larger proportion of the variation in `SalePrice`.

The graph provides a direct visual comparison of both model results.

```

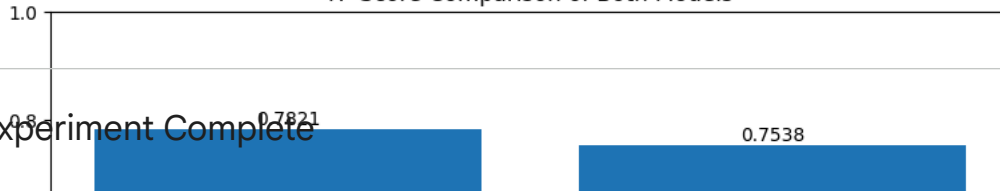
1 # Display the R2 score of both models
2 print("=====")
3 print("      LINEAR REGRESSION MODEL ACCURACY")
4 print("=====")
5
6 print(f"Before Transformation : {baseline_r2:.4f}")
7 print(f"After Log + Scaling   : {transformed_r2:.4f}")
8
9 if transformed_r2 > baseline_r2:
10     print("\nResult: After Log + Scaling performed better.")
11 elif transformed_r2 < baseline_r2:
12     print("\nResult: Before Transformation performed better.")
13 else:
14     print("\nResult: Both models have the same R2 score.")
15
16 # Draw a simple comparison graph
17 model_names = [
18     "Before Transformation",
19     "After Log + Scaling"
20 ]
21
22 r2_scores = [
23     baseline_r2,
24     transformed_r2
25 ]
26
27 plt.figure(figsize=(8, 5))
28
29 bars = plt.bar(model_names, r2_scores)
30
31 plt.title("R2 Score Comparison of Both Models")
32 plt.xlabel("Model")
33 plt.ylabel("R2 Score")
34 plt.ylim(0, 1)
35
36 # Display the exact R2 value above each bar
37 for bar, score in zip(bars, r2_scores):
38     plt.text(
39         bar.get_x() + bar.get_width() / 2,
40         bar.get_height(),
41         f"{score:.4f}",
42         ha="center",
43         va="bottom"
44     )
45
46 plt.tight_layout()
47 plt.savefig("../outputs/model_comparison.png", dpi=150, bbox_inches="tight")
48 plt.show()

```

```
=====
      LINEAR REGRESSION MODEL ACCURACY
=====
Before Transformation : 0.7821
After Log + Scaling   : 0.7538
```

Result: Before Transformation performed better.

R² Score Comparison of Both Models



✓ Experiment Complete

```
Load Data
↓
Explore Data
↓
Select ≤ 10 Features
↓
Handle Missing Values
↓
Visualize + Measure Skewness
↓
Inspect + Treat Suitable Outliers
↓
Train/Test Split
↓
Baseline Linear Regression
↓
R2 + MAE + MSE + RMSE
↓
Log Transformation
↓
StandardScaler
↓
Second Linear Regression
↓
R2 + MAE + MSE + RMSE
↓
Compare
↓
Conclusion
```

Declaration

I hereby declare that the work submitted in this practical assignment is my own and has not been copied from any other source.

Student Name	Shantanu Suryawanshi
PRN	125M1H026
Date	02-09-2026

Prof. Prakash Ukhalkar

Course Teacher - Advanced Data Science Lab [MCA33PE21]

MCA33PE21 Advanced Data Science Lab Practical Assignment 01